

PraxisZeit

Voll-Audit · Security & ArbZG-Compliance

Version 1.4.3

Branch master · HEAD 5e03cdf

Auditdatum: 23. Mai 2026

Letztes Audit: 08. April 2026 · Delta + Voll-Audit

Executive Summary

PraxisZeit befindet sich in **gutem Sicherheitszustand** und ist im On-Prem-Single-Tenant-Betrieb **vollständig ArbZG-konform**. Alle vier HIGH-Findings vom Audit am 08.04. sind nachweislich behoben. Das neue Feature `vacation-request-edit` ist sauber implementiert (Tenant-Scope, `with_for_update`, Audit-Logs, Pydantic-Schemas korrekt).

Findings-Übersicht

Severity	Anzahl	Bereiche
CRITICAL	0	—
HIGH	1	E-Mail HTML-Injection im Signup-Flow (nur SaaS-Modus)
MEDIUM	7	F-026 Inkonsistenz, RLS-Lücke <code>stripe_events</code> , Signup-Passwort-Discrepanz, CI-Abdeckung, PublicHoliday-Tenant-Filter (3 Sub-Items), Tenant-Filter <code>rest_time_service</code>
LOW	10	Tech-Debt: <code>max_length</code> , <code>ge/le-Constraints</code> , <code>Withdraw-Race</code> , <code>Stripe-PII-Excerpt</code> , <code>Error-Logs</code> Tenant-Scope, etc.

Konformität ArbZG (Stand 23.05.)

Paragraph	Status	Delta seit 08.04.
§3 (Tageshöchst Arbeitszeit + 24-Wochen-Ausgleich)	KONFORM	Neu: 24-Wochen-Report <code>/api/admin/reports/24-week-average</code>
§4 (Ruhepausen)	KONFORM	Unverändert
§5 (Mindestruhezeit 11h)	KONFORM, verifiziert	Echtzeit-Warning beim Einstempeln funktioniert
§6 (Nachtarbeit 23:00–06:00)	KONFORM	<code>arbzg_utils.is_night_work()</code> einziger Einstiegspunkt
§9/§10 (Sonn-/Feiertagsruhe)	KONFORM (On-Prem) / TEILWEISE (Multi-Tenant)	7 Stellen mit <code>PublicHoliday</code> ohne <code>tenant_id</code> -Filter
§11 (15 freie Sonntage, Ersatzruhe)	KONFORM	Unverändert
§14 (48h-Wochenwarnung)	KONFORM	Unverändert
§16 (Aufzeichnungspflicht)	KONFORM	VR-Edit/Cancel-Audit-Logs korrekt eingetragen
§18 (Ausnahmen)	KONFORM	Unverändert
EU-AZ-RL Aufzeichnungspflicht (EuGH C-55/18)	KONFORM	Unverändert

Empfehlung in einem Satz

Vor dem nächsten Kunden-Deployment das HIGH-Finding (HTML-Escape im Signup-Mail) fixen und die `_get_*_in_tenant`-Helper für die 10+ F-026-Stellen einführen; **vor dem SaaS-Rollout (Phase 4)** die `PublicHoliday`-Queries und `stripe_events`-RLS härten. Alles andere ist Backlog.

TEIL 1 — ArbZG-Compliance

A. Datenmodell & Persistenz

A-1: Zeitstempel und Granularität — KONFORM

`TimeEntry.start_time` / `end_time` als `time`-Feld, minutengenau nach `replace(second=0, microsecond=0)` in `time_entries.py:240, 305`. Gesetzeskonform (sekundengenau ist nicht vorgeschrieben).

A-2: Audit-Trail und Revisionssicherheit — KONFORM

`TimeEntryAuditLog` wird in allen schreibenden Pfaden befüllt:

Pfad	Datei:Zeile	Source-Marker
Employee-Löschung	<code>time_entries.py:706-716</code>	<code>manual</code>
Admin-Create/Update/Delete	<code>admin_time_entries.py:101-105, 195-207, 240-244</code>	<code>manual</code>
Change-Request-Approval	<code>admin_change_requests.py</code> (3 Typen)	<code>change_request</code>
Absence-CR-Approval	<code>admin_change_requests.py:283-349</code>	<code>absence_request_approval</code>
Auto-Close	<code>time_entries.py:150-168</code>	<code>auto_close</code>
Vacation-Request-Cancel	<code>vacation_requests.py:211-221</code>	<code>vacation_request_cancel</code>
Vacation-Request-Edit (NEU)	<code>vacation_requests.py:349-361</code> , <code>admin_vacations.py:372</code>	<code>vacation_request_edit</code>

Alle Source-Marker liegen unter dem `varchar(40)`-Limit (Migration 037). Längster Wert: `absence_request_approval` (24 Zeichen).

LOW A-2-01 — `vacation_requests.py` enthält keinen Audit-Log-Eintrag beim **Erstellen** eines Urlaubsantrags. Genehmigung, Bearbeitung und Stornierung sind geloggt; der Erstellzeitpunkt fehlt. Für §16 ArbZG nicht explizit vorgeschrieben, erschwert aber Aufsichtsprüfungen.

Fix: Beim `POST /api/vacation-requests/` einen `TimeEntryAuditLog`-Eintrag mit `action="create"`, `source="vacation_request"` anlegen.

A-3: Zeitzonen — KONFORM

`timezone_service.now_local() / today_local()` zentral. §5-Berechnung in `time_entries.py:262-267` nutzt `datetime.combine(..., tzinfo=LOCAL_TZ)` auf beiden Seiten → DST-Übergänge korrekt (F-030).

A-4: 730-Tage-Aufbewahrungspflicht (§16 ArbZG) — KONFORM

`admin_users.py:202-212`: Purge-Schutz prüft `days_since < 730` → HTTP 409 mit Klartext-Begründung. `can_purge`-Flag in Listansicht konsistent (`:127`). Anonymisierung behält Zeiteinträge, löscht nur Abwesenheiten (korrekt — keine Aufbewahrungspflicht für Abwesenheiten nach ArbZG).

B. Geschäftslogik & Validierung

B-1: §3 — Tagesarbeitszeit — KONFORM

Hard-Stop bei 10h und Warnung bei 8h in allen sechs Pfaden:

Pfad	Hard-Stop 10h	Warnung 8h	Wochenwarnung 48h
<code>time_entries.py</code> create (Z. 504)	✓	✓ (Z. 513)	✓ (Z. 515-524)
<code>time_entries.py</code> update (Z. 623)	✓	✓ (Z. 644)	✓ (Z. 646-656)
<code>time_entries.py</code> clock_out (Z. 318)	✓	✓ (Z. 347)	✓ (Z. 349-359)
<code>admin_time_entries.py</code> create (Z. 59)	✓	✓ (Z. 66-67)	✓ (Z. 70-76)
<code>admin_time_entries.py</code> update (Z. 163)	✓	✓ (Z. 170-171)	✓ (Z. 174-180)
<code>change_requests.py</code> create (Z. 214)	✓	— (Antrag)	—
<code>admin_change_requests.py</code> approve (Z. 367-399)	— (Post-Commit Warn)	✓ (Z. 376-399)	✓ (Z. 388-399)

B-2: §3 — 24-Wochen-Ausgleichszeitraum (NEU) — KONFORM

`reports.py:695-772` : Neuer Endpoint `GET /api/admin/reports/24-week-average` berechnet pro aktivem, nicht-exemtem Mitarbeiter das Verhältnis der Gesamtstunden zum Kontingent $8h \times (24 \times work_days_per_week)$. `exempt_from_arbzg` -User werden korrekt übersprungen (`:722`).

NICE-TO-HAVE — Der Report nutzt `float(e.net_hours or 0)` . Falls `net_hours` für historische Einträge `None` ist, werden diese als 0h gezählt → Durchschnitt verzerrt nach unten.

Fix: Direkt `_net_hours(e.start_time, e.end_time, e.break_minutes)` berechnen.

B-3: §4 — Ruhepausen — KONFORM

`break_validation_service.py` implementiert korrekt:

- `>6h` Netto → 30 Min Pause (Z. 87)
- `>9h` Netto → 45 Min Pause (Z. 79)
- §4 Satz 2: Pausenabschnitte ≥ 15 Min (Z. 65-69 für Lücken, Z. 95-99 für deklarierte Pausen)
- Lücken zwischen Einträgen ≥ 15 Min zählen als Pause (Z. 69)
- `exclude_entry_id` korrekt in allen Update-Pfaden

Systemische Einschränkung (keine Verletzung): `break_minutes` ist ein einzelner Integer je `TimeEntry` — wann innerhalb der Schicht die Pause genommen wurde, kann nicht geprüft werden. §4 ArbZG schreibt nur Gesamtdauer vor, nicht den Zeitpunkt.

B-4: §5 — Echtzeit-Ruhezeit-Warnung — KONFORM, verifiziert

`time_entries.py:247-269` im `clock_in()` :

- Lade letzten abgeschlossenen Eintrag (`end_time.isnot(None)` , geordnet absteigend)
- TZ-aware `datetime.combine` auf beiden Seiten (DST-safe)
- Bei `rest_hours < 11` : Warnung `REST_TIME_WARNING: Nur X.Xh Ruhezeit...` in `clock_in_warnings`
- Nur für nicht-exempte User (`:248`)

Frontend zeigt Warning via `showArbZgWarnings(toast, res.data?.warnings)` im `StampWidget.tsx:73`. Retrospektiver Report (`rest_time_service.py:66-75`) berechnet max-End vs. min-Start des Folgetags → behandelt Splitschichten korrekt.

B-5: §6 — Nachtarbeit — KONFORM

`arbzg_utils.is_night_work()` ist kanonischer Einstiegspunkt; importiert von Routern (employee/admin/CR), Reports und Export-Services. Nachtzeit-Definition 23:00–06:00 (entspricht §2 Abs. 4 ArbZG-Normalfenster; alternatives 22:00–05:00 wäre nur per Tarifvertrag, nicht relevant). Nachtarbeit-Report erkennt Nachtarbeitnehmer ab ≥48 Nachtarbeitstagen/Jahr (`reports.py:546-599`).

B-6: §9/§10 — Sonn- und Feiertagsruhe — KONFORM (On-Prem) / TEILWEISE (Multi-Tenant)

`time_entries.py` nutzt in allen schreibenden Pfaden korrekt `is_holiday(db, date, tenant_id=current_user.tenant_id)` (Z. 101, 362, 527, 662). Der Saturday-Bug aus früheren Audits ist gefixt.

MEDIUM B-6-01 — PublicHoliday-Queries ohne tenant_id (7 Stellen, 4 Dateien)

Im Single-Tenant-On-Prem-Betrieb durch RLS abgefangen, im kommenden SaaS-Betrieb (Phase 4) potenzielles Cross-Tenant-Leak: Feiertage eines Tenants werden für anderen verwendet → Arbeitstag fälschlich als Feiertag gewertet (oder umgekehrt).

Datei:Zeile	Code
<code>absences.py:250-252</code>	<code>db.query(PublicHoliday).filter(PublicHoliday.year == year).all()</code>
<code>admin_vacations.py:159</code>	<code>db.query(PublicHoliday).filter(PublicHoliday.year == year).all()</code>
<code>calculation_service.py:201-203</code>	in <code>get_monthly_target()</code>
<code>calculation_service.py:387-390</code>	in <code>get_overtime_account()</code>
<code>calculation_service.py:463-465</code>	in <code>get_ytd_summary()</code>
<code>calculation_service.py:633</code>	in <code>count_workdays()</code>

Fix: Optionalen `tenant_id`-Parameter in den Calculation-Service-Funktionen ergänzen analog `is_holiday()`. Aufrufer pflegen `tenant_id=current_user.tenant_id` durch.

B-7: §18 — Ausnahmen (leitende Angestellte) — KONFORM

`user.exempt_from_arbzg` als erste Bedingung vor §3/§4/§6-Checks in allen Pfaden. Defensive Logik in `admin_time_entries.py:147`: wenn User nicht gefunden, greift trotzdem ArbZG-Check.

C. Benutzeroberfläche & UX

C-1: Warnhinweise im Frontend — TEILWEISE

`showArbzbzWarnings(toast, warnings)` korrekt aufgerufen in `StampWidget.tsx:73, 98` und `TimeTracking.tsx:337`.

LOW C-1-01 — ChangeRequestForm ignoriert Backend-Warnings

`frontend/src/components/ChangeRequestForm.tsx:45-61`: Der `await apiClient.post('/change-requests', ...)` ignoriert die Response. Das Backend gibt bei §6-Nacharbeiter-Verstößen Warnings zurück (`change_requests.py:283-291`), die der Mitarbeiter nie sieht.

```
// Aktuell:
await apiClient.post('/change-requests', { ... });
onSuccess();

// Korrekt:
const response = await apiClient.post('/change-requests', { ... });
showArbzbzWarnings(toast, response.data?.warnings);
onSuccess();
```

C-2: Mitarbeiter-Transparenz (EuGH C-55/18) — KONFORM

`GET /api/time-entries/` liefert eigene Einträge mit `is_editable`-Flag und Warnings. Grundanforderung des EuGH-Urteils erfüllt.

D. Reporting & Export

D-1: Gesetzlich vorgeschriebene Reports — KONFORM

Report	Endpoint	Paragraph	
Ruhezeit-Verstöße	<code>GET /rest-time-violations</code>	§5	
15-freie-Sonntage	<code>GET /sunday-summary</code>	§11	
Nachtarbeit-Übersicht	<code>GET /night-work-summary</code>	§6	
Ersatzruhetage	<code>GET /compensatory-rest</code>	§11	
24-Wochen-Ausgleich (NEU)	<code>GET /24-week-average</code>	§3 Abs. 2	
Monatsreport	<code>GET /monthly</code>	§16	
Excel/ODS/PDF-Export	<code>GET /export, /export-ods, /export-pdf</code>	§16	
Jahresbericht	<code>GET /export-yearly[-classic</code>	<code>-ods]</code>	§16

D-2: §16 Aufzeichnungsvollständigkeit — KONFORM

`time_entry_audit_logs` erfasst alle schreibenden Operationen. DSGVO-Datenzugriffe auf Gesundheitsdaten werden zusätzlich mit `action="health_data_read"` / `source="dsgvo"` protokolliert

(`reports.py:60-70, 134-144, 234-244`).

E. ArbZG-Sicherheit & Datenschutz

E-1: DSGVO Art. 9 — Maskierung `sick → absent` — KONFORM, verifiziert

Endpoint	Datei:Zeile
Kalender	<code>absences.py:87-101</code>
Team-Upcoming	<code>absences.py:133-145</code>
Monatsreport	<code>reports.py:111</code> (<code>sick_hours if include_health_data else 0.0</code>)
Jahresbericht	<code>reports.py:186-187</code>

E-2: Rollenbasierte Zugriffskontrolle — KONFORM

JWT mit `tid`-Claim + `token_version`-Revocation. `require_admin` für Admin-Routes, `require_superuser` für `/api/superadmin/*` (§16-Notfall-Export auch bei deaktivierten Tenants).

E-3: §16 — Tenant-Deaktivierung sperrt Zeitdaten-Zugriff — TEILWEISE (offen seit 08.04.)

`tenant.is_active == False` → 403 für alle User, §16-Export nur via Superadmin-Notfall-Pfad. Im aktuellen On-Prem-Betrieb nicht relevant.

F. Integration & Schnittstellen

F-1: §9 — Feiertagskalender-Integration — KONFORM (On-Prem)

`holiday_service.py`: alle 16 Bundesländer via `workalendar`, tenant-scoped (Z. 69-77). `sync_current_and_next_year` filtert korrekt per `tenant_id` (Z. 220-225). In-Process-Cache nach `(tenant_id, year)` gekapselt (Z. 158-178), Invalidierung bei Schreibzugriffen.

F-2: §5 — Rest-Time-Service Multi-Tenant — MEDIUM

`rest_time_service.py:116`:

```
users = db.query(User).filter(User.is_active == True).all()
```

Kein expliziter `tenant_id`-Filter. RLS schützt zwar, aber CLAUDE.md F-026-Pattern (belt-and-suspenders) verletzt.

Fix: Signatur um optionalen `tenant_id`-Parameter erweitern, in `reports.py` mit `tenant_id=current_user.tenant_id` aufrufen.

TEIL 2 — Security & Funktion

1. Authentication & Authorization

Status	Komponente	Beobachtung
✓ OK	JWT-Algorithmus	HS256 mit SECRET_KEY ≥32 Zeichen, Weak-Indicator-Check (<code>config.py:172-186</code>)
✓ OK	Token-Expiry	Access 30 min, Refresh 7 Tage, <code>token_version</code> -Counter (<code>auth_service.py:147-194</code>)
✓ OK	Token-Versioning	<code>++</code> bei Logout, Passwort-Change, Rolle-Change (<code>admin_users.py:359, 379, 397</code>)
✓ OK	Password-Hashing	<code>bcrypt_sha256</code> (passlib-kompatibel, ohne passlib-Dependency); v=2-Protokoll; Legacy-Hashes opportunistisch migriert (F-041)
✓ OK	Brute-Force-Schutz	5 Versuche / 15 min, OrderedDict-LRU (F-039) verhindert attacker-evicts-victim
✓ OK	Dummy-Hash bei unknown user	Timing-Equalization gegen Enumeration (<code>auth.py:41, 168</code>)
✓ OK	TOTP Replay-Schutz	Per-User-Counter (<code>auth_service.py:220-260</code> , Migration 032)
✓ OK	Cookie-Security	<code>httponly</code> , <code>secure</code> , <code>samesite="lax"</code> , refresh-cookie auf <code>/api/auth/refresh</code> scoped
✓ OK	CSRF	Double-Submit-Cookie, rotiert bei Refresh, exempt nur für login/refresh/health
⚠ MEDIUM	Password-Komplexität-Discrepanz	Admin-Pfad: ≥10 + Upper/Lower/Digit · Self-Service-Signup: nur min_length=8 → Trial-Tenants können schwächere Passwörter setzen
⚠ LOW	Kein Special-Char-Requirement	NIST 800-63B konform (Länge > Komplexität), aber typische Audit-Erwartung nicht erfüllt
–	Kein Passwort-Reset-Flow	Bewusst (kein Email-Reset-Vektor), aber UX-Lücke für Solo-Admins
–	<code>require_superuser</code>	Konsistent angewendet, klare Semantik (User mit <code>tenant_id IS NULL</code> + role=ADMIN)

2. Multi-Tenant Isolation

Status	Befund	Quelle
✓ OK	RLS auf allen Kern-Tabellen	Migration 027 enabled+forced RLS für 12 Tabellen; 033/034 ziehen nach für <code>tenant_invoices</code> , <code>signup_tokens</code>
⚠ MEDIUM	RLS fehlt auf <code>stripe_events</code>	Migration 035: Tabelle mit <code>tenant_id</code> -Spalte, aber keine <code>ENABLE ROW LEVEL SECURITY</code> . Aktuell nur Superadmin schreibt, CLAUDE.md-Pflicht verletzt.
–	<code>signup_audit_log</code> ohne RLS	Bewusst (Migration 034:85, "anonymous requesters") — DSGVO-konform, sollte dokumentiert sein
⚠ MEDIUM	F-026 belt-and-suspenders inkonsistent	RLS schützt, aber expliziter <code>Model.tenant_id == current_user.tenant_id</code> fehlt in 10+ Stellen
✓ OK	Background-Tasks / <code>SessionLocal()</code>	4 Stellen geprüft, alle setzen <code>set_superuser_context(db)</code>
✓ OK	Auth-Middleware Tenant-Setup	<code>middleware/auth.py:86-93</code> , JWT-tid wird gegen DB-tenant_id validiert
✓ OK	Tenant-Context persistiert	<code>database.py:20-32</code> Event-Listener re-applied SET LOCAL nach jeder Tx-Begin (gefixt seit 08.04.)
✓ OK	Cross-tenant Tests	<code>test_cross_tenant_api.py</code> (12), <code>test_tenant_qls.py</code> (13)

MEDIUM-1 Detail — 10+ Stellen ohne expliziten Tenant-Filter:

Datei:Zeile	Endpoint/Funktion
<code>vacation_requests.py:381</code>	Withdraw VR
<code>change_requests.py:340</code>	Withdraw CR
<code>change_requests.py:325</code>	Get CR (employee)
<code>change_requests.py:61, 148</code>	CR-Lookup bei CR-Create
<code>time_entries.py:432, 571, 688</code>	GET/UPDATE/DELETE entry
<code>absences.py:404</code>	Absence DELETE
<code>admin_time_entries.py:236</code>	Admin entry lookup
<code>admin_change_requests.py:152, 170, 178</code>	CR-Approval Entry/Absence-Lookup

Fix: Helper-Pattern wie `_get_user_in_tenant()` aus `admin_users.py:20-38` einführen: `_get_entry_in_tenant()`, `_get_cr_in_tenant()`, `_get_vr_in_tenant()`, `_get_absence_in_tenant()`. RLS bleibt zweite Verteidigungslinie, explizite Filter machen Findings im Review trivial sichtbar.

3. Input-Validation & Injection

Status	Befund	Quelle
✓ OK	Pydantic-Schemas	Alle POST/PUT/PATCH binden Body mit Field-Constraints, Validators, Patterns
✓ OK	Keine String-interpolierten SQL	Nur 2 raw <code>text()</code> (Advisory-Lock, SET LOCAL), beide mit Named-Params
✓ OK	ORM parameterisiert	Keine <code>db.execute(f"...")</code> mit User-Input
✓ OK	XLS-Import File-Upload	5 MB Limit vor Memory-Load, Tenant-Scoping (<code>import_xls.py:55-66, 91-97</code>)
✓ OK	Profile-Picture-Upload	500 KB Limit, Magic-Bytes-Check (nicht Content-Type) (<code>auth.py:484-497</code>)
✓ OK	Request-Size-Limit	2 MB ASGI-Middleware, Chunked-Transfer-tolerant (<code>static_serving.py:52-115</code>)
⚠ LOW	Free-Text ohne <code>max_length</code>	<code>note</code> , <code>reason</code> , <code>proposed_note</code> etc. als <code>Column(Text)</code> +Pydantic ohne Limit → bis 2 MB pro Record möglich
⚠ LOW	<code>hours: float</code> ohne ge/le	<code>vacation_request.py:11, 41</code> akzeptiert negative Werte
⚠ HIGH	E-Mail-HTML-Injection im Signup	<code>mail_service.py:96-102</code> : <code>practice_name</code> (User-Input) und <code>verification_url</code> (Origin-Header) ungeschützt in HTML-Body

HIGH-1 Detail — Mail-Injection-Vektor:

Im Klartext (Pseudo-Code-Schema, exakte Stelle in `mail_service.py:96-102`):

```
html_body = f"<html><body>"
            f"Willkommen bei <strong>{practice_name}</strong>!<br>"
            f"Klicken Sie hier: <a href='{verification_url}'>Bestaetigen</a>"
            f"</body></html>"
```

Bösartiger Signup mit `practice_name = '<><script>...</script>'` injiziert beliebiges HTML/JS in die Verification-Mail. Empfänger ist zwar der Anmelder selbst (vor email-verify), aber **Spear-Phishing-Vektor**: Angreifer registriert mit beliebiger Empfänger-Mail, liefert Custom-HTML.

Fix: `import html; html.escape(practice_name)` + `html.escape(verification_url)` . 3 Zeilen Diff. Nur in SaaS-Modus relevant — On-Prem unkritisch.

4. Sensitive Data & Secrets

Status	Befund	Quelle
✓ OK	SECRET_KEY-Persistenz	Native: <code>config/.secret-key</code> mit restriktiven Permissions; Docker: ENV + Weak-Indicator-Check
✓ OK	Lizenz-Verifikation (Ed25519)	<code>license.py:115-149</code> ; Avalonia-Mirror prüft <code>alg</code> -Header explizit gegen "EdDSA" (<code>LicenseValidator.cs:115-124</code>)
✓ OK	Update-Manifest signiert	Ed25519 + Host-Allowlist verhindert Hijacking
✓ OK	PII-Scrubbing in Error-Logs	UUIDs + E-Mails per Regex (<code>error_log_service.py:16-26</code>)
✓ OK	DSGVO Art. 9	<code>absences.py</code> Maskierung + <code>reports.py</code> Opt-In + Audit-Log
✓ OK	<code>.env</code> nicht im Repo	<code>.gitignore</code> + Platzhalter in <code>.env.example</code>
✓ OK	DB-Credentials	Native: zur Laufzeit <code>secrets.token_hex(32)</code> , in <code>config/.db-credentials</code> mit 0600/ACL
⚠ LOW	<code>_WEAK_ADMIN_PASSWORDS</code> Liste sehr kurz	<code>main.py:37</code> : nur 4 Werte gelistet, schwache 12+-Zeichen rutschen durch
✓ OK	Stripe-Webhook-Verification	<code>stripe.Webhook.construct_event</code> (<code>stripe_service.py:160-173</code>)
⚠ LOW	Stripe-Event-Payload-Excerpt mit PII	<code>stripe_service.py:197</code> : 4000 Zeichen inkl. Customer-Mail/Adresse ohne Scrubbing
✓ OK	DB-User-Trennung	<code>praxiszeit_app</code> (RLS) für Runtime, <code>praxiszeit</code> (Superuser) nur für Migrations

5. Frontend-Security

Status	Befund
✓ OK	Kein unsicheres React-HTML-Injection-Property (0 Treffer im Repo)
✓ OK	URL-Sanitization (<code>ErrorMonitoring.tsx:37-47</code>)
✓ OK	Access-Token nur in Module-Memory (<code>client.ts:7</code>)
✓ OK	CSRF-Header-Mirror (<code>client.ts:42-54</code>)
✓ OK	Service-Worker-Caches gelöscht bei Logout (<code>authStore.ts:87-93</code>)
✓ OK	Content-Security-Policy konsistent zwischen nginx + native FastAPI
✓ OK	Security-Headers (XFO, XCTO, Referrer, HSTS conditional)
✓ OK	HSTS nur wenn HTTPS (F-050) — kein Brick-Risiko bei HTTP-Native-Install
✓ OK	Grafana auf private Netze beschränkt (<code>nginx.conf:53-72</code>)

6. Funktions-Korrektheit (Kern-Workflows)

Status	Befund	Quelle
✓ OK	<code>clock_in</code> / <code>clock_out</code> Race	<code>_get_open_entry(db, user_id, with_lock=True)</code> mit <code>with_for_update()</code>
✓ OK	Stale-Entry Auto-Close in same TX	F-043: kein intermediate commit (<code>time_entries.py:217-221</code>)
✓ OK	Pausen-Validierung	<code>break_validation_service.validate_daily_break</code> mit <code>exclude_entry_id</code>
✓ OK	§5 Echtzeit-Ruhezeitwarnung	<code>time_entries.py:248-269</code> mit DST-safe TZ
✓ OK	VR-Edit Authorization (NEU)	MA: Tenant + Owner + PENDING-Gate (<code>vacation_requests.py:243-260</code>); Admin: Tenant + 404-Mask (<code>admin_vacations.py:263-278</code>)
✓ OK	VR-Edit Mass-Assignment-Schutz	<code>VacationRequestUpdate</code> nur 5 editierbare Felder, kein <code>status</code> / <code>tenant_id</code> / <code>user_id</code>
✓ OK	VR-Edit Race	<code>with_for_update()</code> (<code>vacation_requests.py:251</code> , <code>admin_vacations.py:269</code>)
✓ OK	VR-Edit Audit-Logs	<code>_format_vacation_request_audit_text</code> + <code>source="vacation_request_edit"</code>
✓ OK	CR-Approval Race	F-028: <code>with_for_update()</code> auf CR-Row, Precondition-VOR-Status
✓ OK	CR-Approval Bulk-Atomicity	Per-item Rollback (<code>admin_change_requests.py:438-448</code>)
⚠ LOW	Withdraw VR/CR ohne <code>with_for_update</code>	<code>vacation_requests.py:381</code> , <code>change_requests.py:340</code> — Doppelklick → 404, funktional unschädlich aber inkonsistent
✓ OK	Vacation-Refund bei Sick	Audit-Log VOR Delete
✓ OK	<code>net_hours</code> Floor	<code>max(0.0, ...)</code> (<code>time_entries.py:32-33</code>)
✓ OK	Export Multi-Entry pro Tag	korrekt
✓ OK	Überstundenausgleich	Soll bleibt, Ist=0h (CLAUDE.md-Regel eingehalten)

7. Native-Installer-Spezifisch

Status	Befund
✓ OK	<code>setup.bat</code> PG-Installer-Passwort: 32 Zufalls-Zeichen pro Install, nach Init verworfen
✓ OK	<code>setup.bat</code> Admin-Check (<code>net session</code>)
✓ OK	<code>setup.bat</code> PG-Reuse via Junction; <code>rd /s /q</code> folgt nicht → Uninstaller sicher
✓ OK	<code>setup.bat</code> keine Befehlsinjection (System-Variablen, kein User-Input)
✓ OK	File-Permissions auf Credentials: Windows <code>icacls /inheritance:r</code> + Owner+SYSTEM+S-1-5-32-544 (F-037 locale-independent)
✓ OK	SECRET_KEY: <code>secrets.token_hex(32)</code> = 256-bit
✓ OK	DB-Passwort: <code>secrets.token_hex(32)</code> (Superuser + App-User)
✓ OK	<code>_validate_pg_identifier</code> (Whitelist), <code>_escape_pg_password</code> (Quote-Doppelung)
✓ OK	<code>restore-backup.template.bat</code> fordert "LOESCHEN"-Bestätigung
⚠ LOW	<code>restore-backup.bat</code> Path-Validation: <code>r'%BACKUP_GZ%'</code> raw-string schützt <code>\</code> aber nicht <code>'</code>
✓ OK	Update-Wizard PS1: kein <code>iex</code> / <code>Invoke-Expression</code>
✓ OK	Update-Manifest Signaturprüfung auch im Auto-Update-Pfad
–	Avalonia-Setup: Tests grün (20 Cases), Mirror der Backend-Lizenz-Verifikation inkl. <code>alg</code> -Whitelist

8. Test-Coverage Gap-Analyse

Bereich	Status
Backend pytest	~530 Test-Funktionen
Backend cross-tenant API	12 Cases
Backend RLS-Integration	13 Cases (echtes Postgres)
Backend Concurrency	4+ Cases mit <code>with_for_update</code>
E2E Playwright	22 Specs / 116 Cases
Frontend Vitest	vorhanden

Test-Lücken:

Lücke	Severity
Withdraw-Race (VR/CR DELETE ohne <code>with_for_update</code>)	LOW
Stripe-Webhook Signature-Replay (Idempotenz getestet, Tampering nicht explizit)	LOW
Mail-Service HTML-Escape (bei HIGH-1-Fix mit-testen)	MEDIUM
License-Public-Key-Rotation (Backend+Avalonia-Synchronisation)	LOW
Native <code>restore-backup.bat</code> (nur manueller Pfad)	LOW

MEDIUM CI-Lücke — `.github/workflows/cross-tenant-ci.yml` ist die einzige CI. Sie deckt nur Backend (Unit + RLS) ab. NICHT in CI:

- E2E Playwright (Commit `5e03cdf` erwähnt playwright-CI-Bump, aber Workflow existiert nicht!)
- Frontend tsc / eslint / vitest
- Frontend vite build (Bundle-Größen-Check)
- Docker Compose Build
- Secret-Scanning (TruffleHog, gitleaks)
- Dependency-Vulnerability-Scan (pip-audit, npm audit)

Existiert nur in `scripts/local-ci.sh`. **Fix:** Workflows aus `local-ci.sh` portieren.

TEIL 3 — Priorisierte Action-Items (konsolidiert)

CRITICAL (Release-Blocker)

(keine)

HIGH (vor nächstem Kunden-Deployment)

HIGH-1 — HTML-Injection im Signup-Mail-Body

- **Datei:** `backend/app/services/mail_service.py:96-102`
- **Risiko:** Spear-Phishing-Vektor in SaaS-Modus; Angreifer registriert mit Custom-`practice_name`, beliebige Empfänger-Mail, eingebettetes HTML/JS
- **Fix:** `import html; html.escape(practice_name)` + `html.escape(verification_url)` (3 Zeilen Diff)
- **Test:** Unit-Test mit `practice_name = '<script>alert(1)</script>'`, prüfen dass `<script>` im Output

MEDIUM (vor nächstem Quartal / vor SaaS-Phase 4)

MEDIUM-1 — F-026 belt-and-suspenders Inkonsistenz

- **Dateien:** 10+ Stellen (siehe Tabelle in Teil 2 §2)
- **Risiko:** RLS schützt aktuell, aber Single-Point-of-Failure bei RLS-Bug; CLAUDE.md-Pattern verletzt

- **Fix:** Helper `_get_entry_in_tenant`, `_get_cr_in_tenant`, `_get_vr_in_tenant`, `_get_absence_in_tenant` analog `admin_users._get_user_in_tenant` einführen

MEDIUM-2 — Fehlende RLS auf `stripe_events`

- **Datei:** `backend/alembic/versions/2026_04_24_1600-035_stripe_events.py:23-35`
- **Risiko:** CLAUDE.md-Pflichtregel verletzt; aktuell nur Superadmin-Context-Schreiber, aber Defense-in-Depth fehlt
- **Fix:** Neue Migration 038 mit `ENABLE ROW LEVEL SECURITY` + Policy, analog 033/034

MEDIUM-3 — Signup-Passwort-Komplexität-Discrepanz

- **Datei:** `backend/app/schemas/signup.py:13` vs. `backend/app/schemas/user.py:10-20`
- **Risiko:** Self-Service-Trial-Admins können schwächere Passwörter setzen als per Admin-API
- **Fix:** `_validate_password_complexity` auch in `SignupRequest.admin_password`-Validator anwenden

MEDIUM-4 — CI deckt nur Backend ab

- **Datei:** `.github/workflows/cross-tenant-ci.yml` (einzige Workflow)
- **Risiko:** Recent commit erwähnt playwright-CI, die nicht existiert; regressions slip in
- **Fix:** Workflows aus `scripts/local-ci.sh` portieren (frontend-lint, vitest, build, e2e mit Postgres-Service)

MEDIUM-5 bis 7 — PublicHoliday + rest_time_service Multi-Tenant (ArbZG-Bereich, siehe B-6-01 und F-2)

- 4 Dateien × 7 PublicHoliday-Stellen + 1 × `rest_time_service.py:116`
- **Risiko:** Vor SaaS-Phase 4 unkritisch (RLS schützt im aktuellen On-Prem-Betrieb), danach Pflicht
- **Fix:** Optionalen `tenant_id`-Parameter in Calculation-/Rest-Time-Service-Signaturen ergänzen

LOW (Backlog / Tech-Debt)

LOW-1 — Brute-Force-Tracker per-worker — OrderedDict im Module-Scope; bei N Unicorn-Workern effektiver Lockout $N \times 5/15\text{min}$. **Fix:** Redis-backed slowapi.

LOW-2 — Stripe-Event-Payload-Excerpt mit PII — 4000 Zeichen inkl. Kunden-Mail/Adresse. **Fix:** Excerpt auf nicht-PII-Felder reduzieren oder via `_scrub_pii` jagen.

LOW-3 — Free-Text ohne max_length — `note` / `reason` / `proposed_note` bis 2 MB pro Record. **Fix:** `Field(..., max_length=2000)` auf alle freien Text-Felder.

LOW-4 — `hours: float` ohne ge/le in VR-Schemas — `vacation_request.py:11, 41`. **Fix:** `Field(..., ge=0, le=24)` analog absence.

LOW-5 — Withdraw VR/CR ohne `with_for_update` — `vacation_requests.py:381`, `change_requests.py:340`. **Fix:** `.with_for_update()` analog Edit/Approve.

LOW-6 — `restore-backup.bat` Quote-Injection — `installer/windows/restore-backup.template.bat:95` raw-string-Path. **Fix:** Pfad via `sys.argv` statt String-Interpolation.

LOW-7 — Error-Logs nicht tenant-scoped — `error_log_service.py:96-101` & `routers/error_logs.py:67-76` listen alle Errors deployment-weit. **Fix:** Im SaaS-Modus auf `tenant_id == current_user.tenant_id` filtern.

LOW-8 — Passwort-Komplexität ohne Sonderzeichen — `schemas/user.py:14-19`. NIST/BSI-konform, aber Audit-Erwartung. **Fix:** Optional 4. Constraint oder Begründung in `docs/SECURITY.md`.

LOW-9 — Vacation-Request-Create Audit-Log fehlt — `vacation_requests.py`. **Fix:** `TimeEntryAuditLog` mit `action="create"`, `source="vacation_request"` bei POST.

LOW-10 — 24-Wochen-Report `net_hours`-Berechnung — `reports.py:735` nutzt `e.net_hours or 0`. **Fix:** direkt `_net_hours(e.start_time, e.end_time, e.break_minutes)`.

Verbesserungen seit 08.04.2026

Finding vom 08.04.	Status heute
HIGH: passlib unmaintained + bcrypt 5 broken	✓ Gefixt: F-041 bcrypt_sha256 in-house Implementation, wire-kompatibel
HIGH: Access-Token in localStorage (XSS-Diebstahl)	✓ Gefixt: F-023 Module-Memory only, Refresh via HttpOnly-Cookie
HIGH: User-Enumeration via Login-Timing	✓ Gefixt: F-040 Dummy-bcrypt-Verify bei unknown user
HIGH: Brute-Force ohne Lockout	✓ Gefixt: F-039 5/15min Lockout mit OrderedDict-LRU
MEDIUM: Multi-Tenant App-Layer-Filter inkonsistent	⚠ Teilweise: User-Lookups via <code>_get_user_in_tenant</code> konsolidiert, aber Entry/CR/Absence/VR-Lookups noch offen (MEDIUM-1)
MEDIUM: CR-Approval Race	✓ Gefixt: F-028 <code>with_for_update()</code> + Precondition-VOR-Status
MEDIUM: clock_in/out Race	✓ Gefixt: VULN-009
MEDIUM: Token-Version nicht bei Rolle-Change	✓ Gefixt: VULN-010
MEDIUM: Traceback-Größen-DoS	✓ Gefixt: VULN-012 2000-char Cap
MEDIUM: Grafana ohne IP-Allowlist	✓ Gefixt: VULN-008 (nginx-side)

Bilanz: 4/4 HIGH-Findings vom 08.04. behoben. 5/6 MEDIUM behoben. Eine offene MEDIUM (App-Layer-Filter) ist nun MEDIUM-1 und um konkrete Datei:Zeile-Liste angereichert.

Anhang A — Geprüfte Dateien

Backend

- `app/main.py`, `app/database.py`, `app/config.py`

- `app/middleware/{auth,csrf,license,static_serving}.py`
- `app/routers/{auth,time_entries,absences,vacation_requests,change_requests,admin_users,admin_change_requests,admin_vacations,admin_settings,admin_updates,import_xls,reports,journal,billing,public_signup,superadmin,error_logs}.py`
- `app/services/{auth_service,mail_service,signup_service,stripe_service,xls_import_service,error_log_service,break_validation_service,arbzg_utils,holiday_service,rest_time_service,calculation_service}.py`
- `app/core/{license,updater}.py`
- `app/schemas/{user,vacation_request,signup,absence,change_request,time_entry}.py`
- `alembic/versions/2026_03_25_2000-027_*.py`, `2026_04_24_*-{033,034,035,036}*.py`

Frontend

- `src/api/client.ts`, `src/stores/authStore.ts`
- `src/pages/admin/ErrorMonitoring.tsx`
- `src/components/{StampWidget,ChangeRequestForm}.tsx`
- `src/pages/TimeTracking.tsx`
- `src/utils/arbzgWarnings.ts`
- `nginx.conf`

Installer / Native

- `installer/windows/{setup.bat,restore-backup.template.bat,update-wizard.bat}`
- `installer/setup/src/PraxisZeit.Setup.Core/Services/{LicenseValidator,PraxisZeitConfigWriter,ScriptRunner}.cs`
- `praxiszeit-server.py`

CI / Doku

- `.github/workflows/cross-tenant-ci.yml`
- `scripts/local-ci.sh`
- `docs/superpowers/reports/2026-05-06-vacation-request-edit-review.md`

Anhang B — Audit-Methodik

- **Statische Code-Analyse** mit `grep`, AST-Walks und gezielten File-Reads
- **Schema-Inspektion** der Alembic-Migrationen für RLS-Coverage
- **Endpoint-Mapping** aller schreibenden Routen gegen ArbZG-Paragraphen
- **Cross-Referenzen** zur Memory (`/home/manuel/.claude/.../project_review-2026-04-08.md`) für Delta-Vergleich
- **Verifikation existierender Test-Coverage** über `pytest --collect-only`-Annahmen
- **Severity-Klassifikation** nach OWASP-Schema (CRITICAL=Datenleck/Account-Takeover · HIGH=Compliance-Bruch oder Spear-Phishing-Vektor · MEDIUM=Defense-in-Depth · LOW=Tech-Debt)

Hinweis: Dieser Bericht ist eine technische Code-Analyse ohne Rechtsverbindlichkeit. Bei branchenspezifischen Tarifverträgen (TV-MFA, TV-Ärzte) oder Betriebsvereinbarungen kann die ArbZG-Bewertung einzelner Punkte abweichen. Für verbindliche Rechtsfragen ist die Konsultation eines Fachanwalts für Arbeitsrecht empfohlen.

PraxisZeit Voll-Audit · v1.4.3 · Erstellt am 23. Mai 2026 · Auditor: Claude Opus 4.7 (1M-Kontext) mit ArbZG-Compliance-Sub-Agent (Sonnet 4.6)